OCIR Labs: Parts 2, 3 and 4

Push, Pull and Manage Container Images with Oracle Cloud Infrastructure Registry

Course Topics

- Introduction to Oracle Cloud Infrastructure Registry (OCIR)
- Managing OCIR
- OCIR image concepts
- Demo: Managing OCIR, Part 1 (Create Repository)
- Demo: Managing OCIR, Part 2 (Push Images)
- Demo: Managing OCIR, Part 3 (Pull Images)
- Demo: Managing OCIR, Part 4 (Image Management Tasks)

These labs use OCIR to push, pull and manage container images. This document covers Parts 2, 3 and 4.

Prerequisites

- An OCI tenancy and compartment
- Permission to manage repositories and images
- Docker installed and running
- OCI CLI installed and configured
- An OCI Auth Token
- The tenancy namespace

1. Find your tenancy namespace

```
oci os ns get
```

Example output:

```
{
 "data": "mytenancynamespace"
}
```

Set environment variables:

```
export REGION_KEY=hyd
export NAMESPACE=mytenancynamespace
export REPOSITORY=myapp
export IMAGE_NAME=myapp
export IMAGE_TAG=v1
```

For the Hyderabad region the OCIR regional key is **hyd**, so the registry endpoint is **hyd.ocir.io**. The general format is:

```
<region-key>.ocir.io
```

Cloud Shell Setup: Compartment ID and Variables

If you run the labs in OCI Cloud Shell, Docker and the OCI CLI are already installed and authenticated. Cloud Shell also provides the tenancy OCID in the **$OCI_TENANCY** variable.

Important: --compartment-id must be a compartment OCID (ocid1.compartment.oc1..) or the tenancy OCID (ocid1.tenancy.oc1..). An identity domain OCID (ocid1.domain.oc1..) causes DENIED or Invalid parameter 'compartmentId' errors. If your tenancy has no child compartments, use the tenancy OCID.

Step 1: Find your compartment ID

List all compartments you can access, including nested ones:

```
oci iam compartment list \
  --compartment-id-in-subtree true \
  --access-level ACCESSIBLE \
  --all \
  --query 'data[*].{Name:name, OCID:id, State:"lifecycle-state"}' \
  --output table
```

Copy the OCID (ocid1.compartment.oc1..) of the compartment where your repositories will live. An empty result means everything is in the root compartment, so use the tenancy OCID ($OCI_TENANCY) instead.

Step 2: Set the variables

```
export COMPARTMENT_ID=<compartment-ocid-from-step-1>   # or $OCI_TENANCY
export NAMESPACE=$(oci os ns get --query 'data' --raw-output)
export REGION_KEY=hyd
export REPOSITORY=myapp
export IMAGE_TAG=v1
echo $NAMESPACE
```

Step 3: List repositories

```
oci artifacts container repository list --compartment-id $COMPARTMENT_ID
```

An empty result is fine if no repository exists yet. A permissions error means your user needs a policy such as:

```
Allow group <your-group> to manage repos in tenancy
```

Step 4: Create the repository

If the repository list in Step 3 is empty, create the repository (Part 1 of the demo):

```
oci artifacts container repository create \
  --compartment-id $COMPARTMENT_ID \
  --display-name $REPOSITORY \
  --is-public false
```

Setting --is-public false makes the repository private, so a login with an Auth Token is required to push and pull. Run the list command from Step 3 again to confirm that myapp now appears.

In the lab commands below, use --compartment-id $COMPARTMENT_ID in place of <compartment-ocid>.

Part 2: Push Images to OCIR

Step 1: Create a local Docker image

If you already have an image, skip this step. Create a file named Dockerfile:

```
FROM docker.io/library/nginx
COPY index.html /usr/share/nginx/html/index.html
```

Create an index.html file:

```
<h1>Hello from OCIR</h1>
```

Build the image:

```
docker build -t myapp:v1 .
```

Verify the image:

```
docker images
```

Step 2: Log in to OCIR

Generate an Auth Token from the OCI Console:

1. Open your profile menu.
2. Select My Profile.
3. Open Auth Tokens.
4. Click Generate Token.
5. Copy the token immediately (it is shown only once).

Log in:

```
docker login hyd.ocir.io
```

Prompt	Value
Username	<tenancy-namespace>/<oci-username>
Password	<auth-token>

Example username: mytenancynamespace/alice@example.com

For federated users, the username may include the identity domain:

```
<tenancy-namespace>/<identity-domain>/<username>
```

Step 3: Tag the image for OCIR

OCIR image names use this format:

```
<region-key>.ocir.io/<tenancy-namespace>/<repository>:<tag>
```

Tag the local image:

```
docker tag myapp:v1 hyd.ocir.io/$NAMESPACE/$REPOSITORY:$IMAGE_TAG
```

Verify the tag:

```
docker images
```

Step 4: Push the image

```
docker push hyd.ocir.io/$NAMESPACE/$REPOSITORY:$IMAGE_TAG
```

Example:

```
docker push hyd.ocir.io/mytenancynamespace/myapp:v1
```

A successful push uploads the image layers and produces a digest similar to:

```
sha256:xxxxxxxxxxxxxxxx
```

Step 5: Verify the repository in OCI

Using the OCI CLI:

```
oci artifacts container repository list \
  --compartment-id <compartment-ocid> \
  --display-name myapp
```

List the images:

```
oci artifacts container image list \
  --compartment-id <compartment-ocid> \
  --repository-name myapp
```

Part 3: Pull Images from OCIR

Step 1: Log in to OCIR

If you are not already logged in:

```
docker login hyd.ocir.io
```

Step 2: Pull the image

```
docker pull hyd.ocir.io/$NAMESPACE/$REPOSITORY:$IMAGE_TAG
```

Example:

```
docker pull hyd.ocir.io/mytenancynamespace/myapp:v1
```

Step 3: Verify the downloaded image

```
docker images
```

You should see an entry similar to:

```
hyd.ocir.io/mytenancynamespace/myapp   v1
```

Step 4: Run the image

```
docker run -d --name myapp-container -p 8080:80 \
  hyd.ocir.io/$NAMESPACE/$REPOSITORY:$IMAGE_TAG
```

Test it locally:

```
curl http://localhost:8080
```

Or open http://localhost:8080 in a browser.

Step 5: Pull an image by digest

Tags can be moved to another image, while digests identify an exact image.

```
docker pull hyd.ocir.io/$NAMESPACE/$REPOSITORY@sha256:<image-digest>
```

Example:

```
docker pull hyd.ocir.io/mytenancynamespace/myapp@sha256:xxxxxxxx
```

Part 4: Image Management Tasks

Task 1: List repositories

```
oci artifacts container repository list \
  --compartment-id <compartment-ocid>
```

To include repositories from child compartments, use the appropriate compartment scope and permissions.

Task 2: List images in a repository

```
oci artifacts container image list \
  --compartment-id <compartment-ocid> \
  --repository-name myapp
```

You can also specify the repository in the format <namespace>/<repository-name>:

```
oci artifacts container image list \
  --compartment-id <compartment-ocid> \
  --repository-name mytenancynamespace/myapp
```

Task 3: Push another image version

Build or tag another version:

```
docker tag myapp:v1 hyd.ocir.io/$NAMESPACE/$REPOSITORY:v2
```

Push it:

```
docker push hyd.ocir.io/$NAMESPACE/$REPOSITORY:v2
```

List the repository images again:

```
oci artifacts container image list \
  --compartment-id <compartment-ocid> \
  --repository-name myapp
```

The repository should now contain both v1 and v2.

Task 4: Delete an image by OCID

Step 1: Identify the image OCID

List the images in the repository and copy the OCID of the image you want to delete:

```
oci artifacts container image list \
```

```
  --compartment-id $COMPARTMENT_ID \
  --repository-name myapp \
  --output table
```

Step 2: Delete the image

Delete the image using its OCID:

```
oci artifacts container image delete \
  --image-id <image-ocid>
```

The command normally asks for confirmation. For a lab environment, you can skip it:

```
oci artifacts container image delete \
  --image-id <image-ocid> \
  --force
```

Warning: Deleting an image is permanent. Confirm the correct image OCID before running the command.

Task 5: Delete an image by tag using the repository path

Depending on the OCI CLI version, you can identify an image using the repository and image name:

```
oci artifacts container image delete \
  --repository-name myapp \
  --image-version v1 \
  --compartment-id <compartment-ocid>
```

If this syntax is not accepted by your CLI version, use the image OCID obtained from container image list.

Task 6: Delete the repository

To remove the entire repository:

```
oci artifacts container repository delete \
  --repository-id <repository-ocid>
```

For non-interactive deletion:

```
oci artifacts container repository delete \
  --repository-id <repository-ocid> \
  --force
```

Warning: This can remove all images in the repository. Use it only for the final cleanup step.

Suggested Lab Validation

After completing the exercises, verify that you can:

- [ ] Log in to OCIR using an Auth Token.
- [ ] Tag a Docker image with the correct OCIR path.
- [ ] Push an image to a repository.
- [ ] List repository images.
- [ ] Pull the image from OCIR.

- [ ] Run the pulled image locally.
- [ ] Push a second image tag.
- [ ] Identify an image OCID.
- [ ] Delete a specific image.
- [ ] Delete the repository as a cleanup activity.

Quick Reference

Common OCIR image format:

```
<region-key>.ocir.io/<tenancy-namespace>/<repository>:<tag>
```

Example:

```
hyd.ocir.io/mytenancynamespace/myapp:v1
```

Common mistakes: The most common errors are using the tenancy OCID instead of the tenancy namespace, using an incorrect region key, or using an OCI password instead of an Auth Token.